Backend-process

Gaze_traker.py

```
import cv2
import mediapipe as mp
import numpy as np
from collections import deque
import socket

# ---------------- UDP Setup ----------------
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
SERVER_ADDRESS = ("127.0.0.1", 9999)
last_sent_command = None

# ---------------- MediaPipe Init ----------------
mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
)

# ---------------- Landmarks ----------------
LEFT_IRIS = [474, 475, 476, 477]
RIGHT_IRIS = [469, 470, 471, 472]

LEFT_EYE_CORNERS = [33, 133]
RIGHT_EYE_CORNERS = [362, 263]

LEFT_EYE_EAR = [33, 160, 158, 133, 153, 144]
RIGHT_EYE_EAR = [362, 385, 387, 263, 373, 380]
```

```
# ---------------- Parameters ----------------
EAR_THRESHOLD = 0.18
EYE_CLOSE_FRAMES = 5
SMOOTHING_FRAMES = 7

ratio_buffer = deque(maxlen=SMOOTHING_FRAMES)
close_counter = 0

# ---------------- EAR Function ----------------
def compute_ear(landmarks, eye_indices, w, h):
    pts = [(int(landmarks[i].x * w), int(landmarks[i].y * h)) for i in eye_indices]
    v1 = np.linalg.norm(np.array(pts[1]) - np.array(pts[5]))
    v2 = np.linalg.norm(np.array(pts[2]) - np.array(pts[4]))
    h_dist = np.linalg.norm(np.array(pts[0]) - np.array(pts[3]))
    return (v1 + v2) / (2.0 * h_dist)

# ---------------- Webcam ----------------
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame = cv2.flip(frame, 1)
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    h, w, _ = frame.shape

    results = face_mesh.process(rgb)
    gaze_text = "NO EYE"

    eye_closed = False
```

```
if results.multi_face_landmarks:
    for face_landmarks in results.multi_face_landmarks:

        # -------- Eye Close Detection --------
        left_ear = compute_ear(face_landmarks.landmark, LEFT_EYE_EAR, w, h)
        right_ear = compute_ear(face_landmarks.landmark, RIGHT_EYE_EAR, w, h)

        if left_ear < EAR_THRESHOLD and right_ear < EAR_THRESHOLD:
            close_counter += 1
        else:
            close_counter = 0

        if close_counter >= EYE_CLOSE_FRAMES:
            gaze_text = "CLOSE LOOK"
            ratio_buffer.clear()
            eye_closed = True
            break

        ratios = []

        # -------- LEFT EYE --------
        left_iris = np.array([
            (int(face_landmarks.landmark[i].x * w),
             int(face_landmarks.landmark[i].y * h))
            for i in LEFT_IRIS
        ])
        left_pupil = left_iris.mean(axis=0).astype(int)

        l_lc = int(face_landmarks.landmark[LEFT_EYE_CORNERS[0]].x * w)
        l_rc = int(face_landmarks.landmark[LEFT_EYE_CORNERS[1]].x * w)
```

```
left_ratio = (left_pupil[0] - l_lc) / (l_rc - l_lc)
ratios.append(left_ratio)

cv2.circle(frame, tuple(left_pupil), 4, (0, 255, 0), -1)

# -------- RIGHT EYE --------
right_iris = np.array([
    (int(face_landmarks.landmark[i].x * w),
     int(face_landmarks.landmark[i].y * h))
    for i in RIGHT_IRIS
])
right_pupil = right_iris.mean(axis=0).astype(int)

r_lc = int(face_landmarks.landmark[RIGHT_EYE_CORNERS[0]].x * w)
r_rc = int(face_landmarks.landmark[RIGHT_EYE_CORNERS[1]].x * w)

right_ratio = (right_pupil[0] - r_lc) / (r_rc - r_lc)
ratios.append(right_ratio)

cv2.circle(frame, tuple(right_pupil), 4, (0, 255, 0), -1)

# -------- GAZE DECISION --------
if not eye_closed:
    avg_ratio = sum(ratios) / len(ratios)
    ratio_buffer.append(avg_ratio)
    smooth_ratio = sum(ratio_buffer) / len(ratio_buffer)

    if smooth_ratio < 0.42:
        gaze_text = "LEFT LOOK"
    elif smooth_ratio > 0.58:
        gaze_text = "RIGHT LOOK"
    else:
```

```
                gaze_text = "FORWARD LOOK"

    # -------- SEND COMMAND (LOW LATENCY) --------
    if gaze_text != last_sent_command:
        sock.sendto(gaze_text.encode(), SERVER_ADDRESS)
        last_sent_command = gaze_text

    cv2.putText(
        frame,
        gaze_text,
        (30, 45),
        cv2.FONT_HERSHEY_SIMPLEX,
        1.2,
        (0, 255, 0),
        3
    )

    cv2.imshow("High Efficiency Eye Gaze Tracking", frame)

    if cv2.waitKey(1) & 0xFF == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

Wheelchair_simulation.py

```
import cv2
import mediapipe as mp
import numpy as np
from collections import deque
import socket
```

```
# ---------------- UDP Setup ----------------
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
SERVER_ADDRESS = ("127.0.0.1", 9999)
last_sent_command = None

# ---------------- MediaPipe Init ----------------
mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
)

# ---------------- Landmarks ----------------
LEFT_IRIS = [474, 475, 476, 477]
RIGHT_IRIS = [469, 470, 471, 472]

LEFT_EYE_CORNERS = [33, 133]
RIGHT_EYE_CORNERS = [362, 263]

LEFT_EYE_EAR = [33, 160, 158, 133, 153, 144]
RIGHT_EYE_EAR = [362, 385, 387, 263, 373, 380]

# ---------------- Parameters ----------------
EAR_THRESHOLD = 0.18
EYE_CLOSE_FRAMES = 5
SMOOTHING_FRAMES = 7

ratio_buffer = deque(maxlen=SMOOTHING_FRAMES)
close_counter = 0
```

```
# ---------------- EAR Function ----------------
def compute_ear(landmarks, eye_indices, w, h):
    pts = [(int(landmarks[i].x * w), int(landmarks[i].y * h)) for i in eye_indices]
    v1 = np.linalg.norm(np.array(pts[1]) - np.array(pts[5]))
    v2 = np.linalg.norm(np.array(pts[2]) - np.array(pts[4]))
    h_dist = np.linalg.norm(np.array(pts[0]) - np.array(pts[3]))
    return (v1 + v2) / (2.0 * h_dist)

# ---------------- Webcam ----------------
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame = cv2.flip(frame, 1)
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    h, w, _ = frame.shape

    results = face_mesh.process(rgb)
    gaze_text = "NO EYE"

    eye_closed = False

    if results.multi_face_landmarks:
        for face_landmarks in results.multi_face_landmarks:

            # -------- Eye Close Detection --------
            left_ear = compute_ear(face_landmarks.landmark, LEFT_EYE_EAR, w, h)
            right_ear = compute_ear(face_landmarks.landmark, RIGHT_EYE_EAR, w, h)
```

```
if left_ear < EAR_THRESHOLD and right_ear < EAR_THRESHOLD:
    close_counter += 1
else:
    close_counter = 0

if close_counter >= EYE_CLOSE_FRAMES:
    gaze_text = "CLOSE LOOK"
    ratio_buffer.clear()
    eye_closed = True
    break

ratios = []

# -------- LEFT EYE --------
left_iris = np.array([
    (int(face_landmarks.landmark[i].x * w),
     int(face_landmarks.landmark[i].y * h))
    for i in LEFT_IRIS
])
left_pupil = left_iris.mean(axis=0).astype(int)

l_lc = int(face_landmarks.landmark[LEFT_EYE_CORNERS[0]].x * w)
l_rc = int(face_landmarks.landmark[LEFT_EYE_CORNERS[1]].x * w)

left_ratio = (left_pupil[0] - l_lc) / (l_rc - l_lc)
ratios.append(left_ratio)

cv2.circle(frame, tuple(left_pupil), 4, (0, 255, 0), -1)

# -------- RIGHT EYE --------
right_iris = np.array([
    (int(face_landmarks.landmark[i].x * w),
```

```
                int(face_landmarks.landmark[i].y * h))
               for i in RIGHT_IRIS
            ])
            right_pupil = right_iris.mean(axis=0).astype(int)

            r_lc = int(face_landmarks.landmark[RIGHT_EYE_CORNERS[0]].x * w)
            r_rc = int(face_landmarks.landmark[RIGHT_EYE_CORNERS[1]].x * w)

            right_ratio = (right_pupil[0] - r_lc) / (r_rc - r_lc)
            ratios.append(right_ratio)

            cv2.circle(frame, tuple(right_pupil), 4, (0, 255, 0), -1)

            # -------- GAZE DECISION --------
            if not eye_closed:
                avg_ratio = sum(ratios) / len(ratios)
                ratio_buffer.append(avg_ratio)
                smooth_ratio = sum(ratio_buffer) / len(ratio_buffer)

                if smooth_ratio < 0.42:
                    gaze_text = "LEFT LOOK"
                elif smooth_ratio > 0.58:
                    gaze_text = "RIGHT LOOK"
                else:
                    gaze_text = "FORWARD LOOK"

# -------- SEND COMMAND (LOW LATENCY) --------
if gaze_text != last_sent_command:
    sock.sendto(gaze_text.encode(), SERVER_ADDRESS)
    last_sent_command = gaze_text
cv2.putText(
    frame,
```

```
        gaze_text, (30, 45),
        cv2.FONT_HERSHEY_SIMPLEX,
        1.2,
        (0, 255, 0),
        3)
    cv2.imshow("High Efficiency Eye Gaze Tracking", frame)

    if cv2.waitKey(1) & 0xFF == 27:
        break
cap.release()
cv2.destroyAllWindows()
```

←-----Documentation Scrolling-----→

Document_scroll.py

```
import cv2
import numpy as np
import pyautogui
from tensorflow.keras.models import load_model

# Load trained model
model = load_model("eye_model.h5")

# Start webcam
cap = cv2.VideoCapture(0)

print("Press 'q' to exit")
import time

# Variables (add before loop)
last_action_time = 0
cooldown = 1.0  # seconds
history = []
```

```
history_size = 5

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Flip frame for natural view
    frame = cv2.flip(frame, 1)

    # Resize and preprocess
    img = cv2.resize(frame, (64, 64))
    img = img / 255.0
    img = np.expand_dims(img, axis=0)

    # Prediction
    pred = model.predict(img, verbose=0)[0][0]

    # FIXED LOGIC
    if pred >= 0.50:
        label = "UP"
    elif pred <= 0.35:
        label = "DOWN"
    else:
        label = "CENTER"

    # Store history
    history.append(label)

    if len(history) > history_size:
        history.pop(0)
```

```
    # Check stability
    if history.count("UP") == history_size:
        stable_label = "UP"
    elif history.count("DOWN") == history_size:
        stable_label = "DOWN"
    else:
        stable_label = "CENTER"

    # Cooldown check
    current_time = time.time()

    if current_time - last_action_time > cooldown:
        if stable_label == "UP":
            pyautogui.scroll(20)
            last_action_time = current_time
        elif stable_label == "DOWN":
            pyautogui.scroll(-20)
            last_action_time = current_time

    # Display result
    cv2.putText(frame, f"Movement: {label}", (20, 50),
                cv2.FONT_HERSHEY_SIMPLEX, 1,
                (0, 255, 0), 2)

    cv2.imshow("Eye Movement Detection", frame)

    # Exit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```